

数据驱动

同一套操作步骤、不同的输入数据，跑出多组验证 —— 这就是数据驱动。ATP 提供了轻量的参数化能力，但**刻意做得不强**，本篇会说明为什么。

变量语法

引用方式

在 `input\_data`、`expected`、`locator\_value` 三个字段里可以用 `\${变量名}` 引用变量：

```
...
input_data: ${username}
expected:   ようこそ、${display_name} さん
locator_value: //tr[td[text()="${order_no}"]]
...
```

变量名只允许字母、数字和下划线，区分大小写。

未定义的变量会导致案例在**执行前**校验失败，而不是执行到那一步才报错 —— 这样能更早发现问题。

三种变量来源

按优先级从高到低：

来源	作用域	典型用途
---	---	---
数据集行	当前这一轮执行	参数化的测试数据
案例变量	单条案例	该案例专用的固定值
环境变量	整个项目	环境地址、公共账号

同名时高优先级覆盖低优先级。这个规则让「大部分用环境默认值、个别案例覆盖」的写法成为可能。

环境变量

用途与配置

环境变量在「项目配置 环境管理」里维护，每个环境一套。典型的几个：

```
...
${base_url}    被测系统地址
```

```

${admin_user}    管理员账号
${test_user}     普通测试账号
...

```

案例里一律写`\${base\_url}/login`而不是写死地址，这样同一批案例可以在开发环境、测试环境、预发环境分别执行，切换只需要在执行时选环境。

****密码等敏感信息不要放环境变量****，它对所有能看到项目的人可见。用凭据管理（下节）。

凭据引用

密码类数据用`\${cred{名称}}`引用，值存在独立的凭据库里，在案例编辑界面和执行日志中都显示为`\*\*\*\*\*`：

```

...
input_data: @cred{test_user_password}
...

```

凭据只有`OWNER`角色能维护。执行日志里不会出现明文，报告截图里如果不小心把密码输进了明文框，平台也会对该区域做模糊处理。

数据集

绑定数据集

案例可以绑定一个数据集，数据集是一张二维表，第一行是变量名：

```

| username | password | expect_msg |
|---|---|---|
| `user_a` | `${cred{pwd_a}}` | ログイン成功 |
| `user_b` | `${cred{pwd_b}}` | ログイン成功 |
| `locked_user` | `${cred{pwd_locked}}` | アカウントがロックされています |

```

绑定后，这条案例会执行3轮，每轮取一行。报告里3轮结果分开展示，某一行失败不影响其他行。

数据集不宜过大

****建议单个数据集不超过20行。****

超过这个量级说明测试意图可能有问题——数据驱动是为了覆盖****有代表性的等价类****，不是为了穷举。100行数据跑同一套步骤，99行都在重复验证同一条路径，只是把回归时间拉长了100倍。

正确的做法是挑边界值和典型异常：正常值、空值、超长值、特殊字符、越权账号。五六行往往就够了。

步骤间传递数据

提取与复用

某些场景需要把页面上的值取出来，供后续步骤使用 ——
比如下单后拿到订单号，再用它去列表页查。

用 `ASSERT\_TEXT` 的**捕获模式**：在 `expected` 里写
`->变量名`，平台会把元素的文本存进变量而不做断言：

```
...  
seq 5: ASSERT_TEXT //span[@data-testid="order-no"] expected: ->order_no  
seq 8: ASSERT_TEXT //tr[td[text()="${order_no}"]]/td[@data-col="status"] expected: 支払い待ち  
...
```

捕获的变量作用域是**当前这一轮执行**，案例结束就销毁，不会带到下一条案例。

不能跨案例传递

ATP **不支持案例之间传递数据**。案例 A 里捕获的变量，案例 B 读不到。

这和「案例之间没有依赖关系」是同一个设计取舍（见《案例结构》）。允许跨案例传值就等于引入了执行顺序依赖，会让案例无法单独重跑。

需要共享数据时，正确做法是在案例内部把前置数据造出来，或者用固定的测试数据。

为什么参数化能力做得这么弱

有意的限制

看完上面你可能会觉得 ATP 的数据驱动能力很有限 ——
没有循环、没有条件分支、没有函数、不能算表达式。

这是有意的。ATP 的使用者是不写代码的测试工程师，**每增加一分表达能力，就增加一分「案例变成程序」的风险**。而案例一旦变成程序，就需要有人维护这些程序，平台的意义也就没有了。

判断标准很简单：如果一个需求需要在案例里表达「如果.....就.....」，那它应该是**两条案例**，而不是一条带分支的案例。

复杂场景怎么办

确实存在参数化搞不定的场景，比如需要先调接口造数据。处理方式：

1. **数据准备接口** —— 平台组提供了一批常用的数据准备接口（建订单、重置账号状态等），在「项目配置 数据准备」里配置，会在案例执行前自动调用

2. ****固定测试数据**** —— 在测试环境预置一批稳定的数据，案例直接用

3. ****拆案例**** —— 把造数据的过程本身写成一条 UI 案例

优先用第 2 种。固定数据虽然「不够动态」，但它带来的稳定性收益远大于灵活性损失 —— 而稳定性正是自动化回归最稀缺的东西。